Object Detection with YOLO11: what changed

Simone Bonato · 9 November 2026

Your YOLOv7 workbook, modernized as you asked: the same storyline (setup, data, training, predictions, TP/FP/FN, optional experiments) and your tasks with Sample Answers, on YOLO11 and the Ultralytics Python API, with a new dataset and two new interactive parts. Each page shows your version on the left and the new one on the right; the red numbers mark the exact place.

Is it shorter? In cells and commands, yes: 66 cells instead of 90, 1 shell command instead of 23. In reading, it is longer: about 2,300 visible words instead of 1,400 (1.6 times), because the precision–recall, AP and NMS parts are new and every task keeps your full structure (Before running the code, Explore, Observe and interpret). The Sample Answers are hidden in toggles.

Tested: runs top to bottom headless (CPU smoke test, every check passing) and once in full on a Mac: YOLO11n trained 20 epochs in about 4 minutes there, mAP@50 0.86, mAP@50–95 0.54. A full run in Colab on a T4 is still to do. Why YOLO11 and not the newer YOLO26: on Ultralytics' own 37-dataset fine-tuning benchmark YOLO11n scores slightly higher (46.4 vs 44.4 mAP@50–95), and its classic pipeline shows the NMS step the slider explores.

Your brief, point by point

19 points: 13 done, 6 partly, 0 not done. 'See' names the numbered before/after page that shows it.

You asked	Status	What was done	See
Update to YOLO11 (or another one you suggest)	✅ done	YOLO11n through Ultralytics, pinned to 8.4.208. YOLO26 is newer, but YOLO11n scores slightly higher on Ultralytics' own 37-dataset fine-tuning benchmark (46.4 vs 44.4 mAP@50–95) and keeps the classic NMS step the slider shows.	1
Simplify the setup using the Python API	✅ done	Training, validation and prediction are one Python call each. Shell commands: 23 in your version, 1 now (the install). No runtime pin, no torch reinstall, no git clone, no Drive, no W&B login.	1
Ensure compatibility with Google Colab	⚠️ partly	Not run in Colab yet: Simone does a T4 run next. Tested instead with a headless run on CPU (every check passing) and one full run on a Mac GPU (45 min, 0 errors). Colab now ships Python 3.13; the tests ran on 3.12.	–
A public dataset, about 500 images, a few classes, not COCO-like, with boxes, suitable for fine-tuning	✅ done	Grape bunches in a vineyard (Embrapa WGISD, CC BY-NC 4.0, non-commercial teaching allowed): 1 class, 242 training and 58 validation images. Fine-tuned YOLO11n reaches mAP@50 0.86.	2
Download from Kaggle or so	⚠️ partly	From the dataset's own GitHub page instead. Kaggle needs an account and an API key inside Colab; GitHub works without a login (about 300 MB, one to two minutes).	2
Keep dataset exploration minimal	✅ done	One image browser and the label-format recap. Your two analysis tasks are left to Know Your Images.	3
Keep explanations, student tasks and the IoU / TP / FP / FN exercises; avoid restructuring	✅ done	Same order as yours: setup, model, data, training, predictions, TP/FP/FN, optional experiments. The TP/FP/FN task, the training-settings task, the label-format recap and both optional experiments are kept. Every one of your exercises is on the next page.	4–5, 9
The IoU calculation can be deleted	✅ done	Deleted. IoU is defined in one sentence and points to the optional IoU task in Know Your Images.	5
Interactive precision–recall exercise with YOLO predictions	✅ done	Confidence slider: one image with its TP/FP/FN and the precision–recall curve of each class with the current point.	6
… understand AP	✅ done	AP drawn as the shaded area under each class's curve, checked against Ultralytics' value.	7
… interpret mAP@50 vs mAP@50–95	✅ done	AP at the ten IoU thresholds; their mean is mAP@50–95. A task asks what each rewards (finding vs tight boxes).	7
Optionally compare YOLO11n and YOLO11s	✅ done	Optional, behind one checkbox. In our run YOLO11s was clearly better (mAP@50 0.90 vs 0.86) and took about 1.5 times as long.	9
All cells run sequentially in a fresh runtime	⚠️ partly	Yes on a fresh local kernel (CPU smoke test and full Mac run). Colab still to do (see above).	–
American English	✅ done	An automatic check for British spellings runs with every test; headings in Title Case.	–
Retain sample solutions	✅ done	9 tasks, 9 Sample Answers, written from what the real runs showed.	4–9
Keep training times manageable	⚠️ partly	Default model: about 4 min for 20 epochs on a Mac GPU. Not yet measured on a Colab T4.	–
If possible, a slider for the NMS threshold	⚠️ partly	Built. Duplicates show clearly at 0.95 (false positives 2 → 58). Lost neighbors at low values do not show on this dataset (no bunch is lost at 0.3); the Sample Answer explains it.	8
Keep the notebook short	⚠️ partly	Fewer cells (66 vs 90) and 1 shell command vs 23, but more to read: about 2,300 visible words vs 1,400 (1.6×), because the new precision–recall, AP and NMS parts are explained and every task keeps your full structure. The Sample Answers are hidden in toggles.	–
Complement the exploration notebook without duplicating it	✅ done	No exploration tasks and no IoU implementation here; both are in Know Your Images.	3

Your exercises, one by one

Exercise in your YOLOv7 workbook	In the new notebook
Analyze the data qualitatively (visual inspection)	Cut: Know Your Images covers it. One short image browser remains.
Analyze the data quantitatively (with code)	Cut: Know Your Images covers it.
Choose training parameters (batch size, epochs)	Kept: two sliders and a Sample Answer, then a new task on reading the training curves.
Evaluate the training in wandb.ai	Replaced: the training curves are drawn in the notebook (new task "Is the Model Still Learning?").
Check individual predictions; set the weights path and confidence threshold	Kept in spirit: predictions are browsed image by image; the threshold became the confidence slider.
Find the experiment folder of your run	Dropped: the Python API returns the results, so no folder hunting is needed.
TP, FP, FN in the sample image	Kept, with every box colored by the matching rule.
Calculate the IoU (coding)	Deleted, as you suggested.
Optional: freeze layers during training	Kept as an optional task (freeze=10, the backbone).
Optional: train the tiny model and compare in wandb	Became YOLO11n vs YOLO11s, compared in a table in the notebook.
(new) What does the COCO model see before fine-tuning?	New first task: motivates fine-tuning.
(new) Confidence threshold, AP, mAP@50 vs mAP@50–95, NMS threshold	New tasks from your brief.

1 · Setup: from version pins to one install

Your workbook had to pin an old Colab runtime (Python 3.11, 2025.07), reinstall torch 2.5.0 and clone the YOLOv7 repository. Now one cell installs Ultralytics (pinned 8.4.208) on Colab's current runtime, and training, validation and prediction are each one Python call. Drive and Weights & Biases are no longer needed; the training curves are drawn in the notebook.

Your YOLOv7 workbook

1 The current configuration runs only on Python 3.11. Therefore, we need to select an older runtime in Colab. Runtime → Change runtime type → Runtime version → 2025.07 → Save

```
import torch
print(torch.__version__)
```

```
2.8.0+cu126
```

```
!pip uninstall -y torch torchvision torchaudio
```

```
Found existing installation: torch 2.8.0+cu126
Uninstalling torch-2.8.0+cu126:
  Successfully uninstalled torch-2.8.0+cu126
Found existing installation: torchvision 0.23.0+cu126
Uninstalling torchvision-0.23.0+cu126:
  Successfully uninstalled torchvision-0.23.0+cu126
Found existing installation: torchaudio 2.8.0+cu126
Uninstalling torchaudio-2.8.0+cu126:
  Successfully uninstalled torchaudio-2.8.0+cu126
```

```
!pip install torch==2.5.0 torchaudio==2.5.0 torchvision==0.20.0
```

2
```
Collecting torch==2.5.0
  Downloading torch-2.5.0-cp312-cp312-manylinux1_x86_64.whl.metadata (28 kB)
Collecting torchaudio==2.5.0
  Downloading torchaudio-2.5.0-cp312-cp312-manylinux1_x86_64.whl.metadata (6.4 kB)
Collecting torchvision==0.20.0
  Downloading torchvision-0.20.0-cp312-cp312-manylinux1_x86_64.whl.metadata (6.1 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch==2.5.0) (3.20.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from torch==2.5.0) (4.15.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch==2.5.0) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch==2.5.0) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch==2.5.0) (2025.3.0)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch==2.5.0)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch==2.5.0)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch==2.5.0)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch==2.5.0)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch==2.5.0)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch==2.5.0)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.5.147 (from torch==2.5.0)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch==2.5.0)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparse-cu12==12.3.1.170 (from torch==2.5.0)
  Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-nccl-cu12==2.21.5 (from torch==2.5.0)
  Downloading nvidia_nccl_cu12-2.21.5-py3-none-manylinux2014_x86_64.whl.metadata (1.8 kB)
Collecting nvidia-nvtx-cu12==12.4.127 (from torch==2.5.0)
  Downloading nvidia_nvtx_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.7 kB)
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch==2.5.0)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting triton==3.1.0 (from torch==2.5.0)
  Downloading triton-3.1.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.3 kB)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch==2.5.0) (75.2.0)
Collecting sympy==1.13.1 (from torch==2.5.0)
  Downloading sympy-1.13.1-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision==0.20.0) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision==0.20.0) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy==1.13.1->torch==2.5.0) (1.3.
0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch==2.5.0) (3.0.3)
Downloading torch-2.5.0-cp312-cp312-manylinux1_x86_64.whl (906.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 906.4/906.4 MB 1.3 MB/s eta 0:00:00
Downloading torchaudio-2.5.0-cp312-cp312-manylinux1_x86_64.whl (3.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.4/3.4 MB 2.1 MB/s eta 0:00:00
Downloading torchvision-0.20.0-cp312-cp312-manylinux1_x86_64.whl (7.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.2/7.2 MB 75.0 MB/s eta 0:00:00
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (363.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 363.4/363.4 MB 4.8 MB/s eta 0:00:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (13.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.8/13.8 MB 75.1 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (24.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 66.6 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 57.7 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 3.0 MB/s eta 0:00:00
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.5/211.5 MB 4.8 MB/s eta 0:00:00
Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl (56.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 44.5 MB/s eta 0:00:00
Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl (127.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 MB 21.8 MB/s eta 0:00:00
Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl (207.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 207.5/207.5 MB 13.3 MB/s eta 0:00:00
Downloading nvidia_nccl_cu12-2.21.5-py3-none-manylinux2014_x86_64.whl (188.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 188.7/188.7 MB 14.8 MB/s eta 0:00:00
Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (21.1 MB)
```

New

Setup

1 The next cell installs Ultralytics (the library behind YOLO11), checks for a GPU, and loads the helpers. It takes about a minute.

```
# ▸ Setup (run me first)
```

```
Apple GPU (MPS) found: training runs on it.
Colab: False · smoke mode: False · device: mps
```

New

YOLO11 comes in five sizes: larger models are more accurate but need more memory and time. These are Ultralytics' COCO results at 640 px (mAP@50–95 is explained later).

3

Model	Parameters	Compute (GFLOPs)	COCO mAP@50–95	Typical target
YOLO11n	2.6 M	6.5	39.5	phones, edge devices
YOLO11s	9.4 M	21.5	47.0	small GPUs
YOLO11m	20.1 M	68.0	51.5	consumer GPUs
YOLO11l	25.3 M	86.9	53.4	cloud GPUs
YOLO11x	56.9 M	194.9	54.7	high-end cloud GPUs

We fine-tune **YOLO11n**, the smallest, so training takes minutes. The optional experiments compare it with YOLO11s.

Your YOLOv7 workbook

Download YOLO Repository

3

```
# only needs to be executed once or in case an update is desired/required
# this is the official YOLOv7
!git clone https://github.com/WongKinYiu/yolov7.git
```

```
Cloning into 'yolov7'...
remote: Enumerating objects: 1197, done.
remote: Total 1197 (delta 0), reused 0 (delta 0), pack-reused 1197 (from 1)
Receiving objects: 100% (1197/1197), 74.29 MiB | 18.49 MiB/s, done.
Resolving deltas: 100% (511/511), done.
```

2 · A new dataset, from outside COCO

Cattle from a drone became grape bunches in a vineyard (Embrapa WGISD, CC BY-NC 4.0, so free for teaching): 242 training and 58 validation images, one class. The bunches are large and hang side by side, which the NMS part needs. The images come straight from the dataset's GitHub page (about 300 MB, no login). A new first task shows what the COCO model predicts before fine-tuning, which motivates the training.

Your YOLOv7 workbook

1 For this exercise, we are working with the cattle dataset (data, official website). The cattle dataset was created to detect and count cattle using *unmanned aerial vehicle* (UAV) images. For this exercise, cattle dataset1 (653 images) was used as training data, and dataset2 (14 images) as validation data. To keep the training times shorter during the deep dive, we downsized the images to 800x600px.

Further image preprocessing steps such as scaling of the pixel values could be applied, as illustrated in the segmentation workbook. For simplicity, we are not performing these preparation steps here.

New

The Dataset

Download the dataset, recall YOLO's label format, and look at some images and boxes.

1 A vineyard estimates its harvest by counting grape bunches in photos of each vine row. Bunches hang side by side and hide each other, so a detector must separate neighbors.

Our dataset has vine photos with one box per grape bunch: a single class. A bunch is typically 5 to 10 % of the image width, and neighboring boxes often touch. Dataset exploration is in *Know Your Images*; we only look briefly here.

The next cell downloads the images (about 300 MB, one to two minutes the first time; later runs reuse them) and shrinks each to 640 pixels on its longer side. We use the dataset's own split: 242 training and 58 validation images.

New

```
# ▸ 🔎 Browse Training Images with Their Boxes
```

New

2 🟢 Before Fine-Tuning: What Does a COCO Model See?

The pretrained YOLO11n knows COCO's 80 everyday classes: people, cars, dogs, cups. None is one of ours.

Before running the code:

- What will it predict on our images: nothing, wrong classes, or our objects under another name?

Explore:

- Run the cell to see its predictions on four validation images.

Observe and interpret:

- Which COCO classes does it use, if any? Why do we need to fine-tune?

```
# ▸ 🔎 Predict with the COCO-Pretrained Model
```

Predictions of the COCO-pretrained YOLO11n before fine-tuning (confidence ≥ 0.25)

3 · Dataset exploration: one short look

Your qualitative and quantitative analysis tasks are covered by Know Your Images, so here they are replaced by one sentence and one image browser.

Your YOLOv7 workbook

Analyze the Data Qualitatively

Exercise

1. Analyze the images and labels by visual inspection and report the problems.
2. Get a qualitative sense of the data and try to find any sort of patterns that are visible to the human brain

Your YOLOv7 workbook

Analyze the Data Quantitatively

Exercise

2. Analyze the images and labels using code. E.g, count number of images, images shapes and their frequencies, classes per image, numbers and sizes of object boxes, average +/- std of number of pixels, pixel value ranges and distributions, imbalances, bias, and other quantitative measures that you can think of.

New

The Dataset

Download the dataset, recall YOLO's label format, and look at some images and boxes.

1. A vineyard estimates its harvest by counting grape bunches in photos of each vine row. Bunches hang side by side and hide each other, so a detector must separate neighbors.

1. Our dataset has vine photos with one box per grape bunch: a single class. A bunch is typically 5 to 10 % of the image width, and neighboring boxes often touch. Dataset exploration is in *Know Your Images*; we only look briefly here.

4 · Training: your batch-size and epochs task kept

Same task, now as two sliders with a Sample Answer, followed by the training curves (loss and validation mAP per epoch) and a task on reading them.

Your YOLOv7 workbook

```
# Task: choose training params (see below)
# Task: choose YOLO config params, e.g., augmention, IOU threshold, see file hyp.scratch.cattle.yaml
#### BEGIN SOLUTION
batch_size = ...
epochs = ...
### END SOLUTION
```

Sample Solution

```
#### BEGIN SOLUTION
batch_size = 16
epochs = 10
### END SOLUTION
```

Train

```
# adjust epochs to get a satisfying result
!python train.py \
  --device 0 \
  --workers 8 \
  --batch-size $batch_size \
  --epochs $epochs \
  --img 640 640 \
  --data {data_dir}/cattle.yaml \
  --hyp {data_dir}/hyp.scratch.cattle.yaml \
  --cfg cfg/training/yolov7x.yaml \
  --weights yolov7x.pt \
  --name yolov7x-cattle
```

New

🟢 Training Settings: How Long and How Big?

YOLO was trained for hundreds of epochs on COCO's 118,000 images. We have a few hundred images, a pretrained model, and a free Colab GPU.

Before running the code:

- Will you need more or fewer epochs than for COCO? Why?

Explore:

1. Set `EPOCHS` (5 to 50) and `BATCH_SIZE` (8, 16 or 32; a T4 handles 16) in the cell below. Each extra 10 epochs costs about two minutes on a T4.
2. Run the training cell after it.

Observe and interpret:

- Did the model stop improving before the last epoch, or would more epochs help?

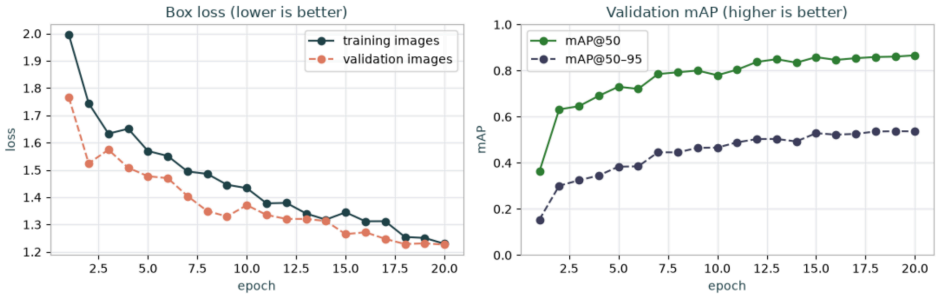
```
# ✎ YOUR TURN: training settings
EPOCHS = 20  # @param {type:"slider", min:5, max:50, step:5}
BATCH_SIZE = 16  # @param [8, 16, 32] {type:"raw"}
IMG_SIZE = 640  # pixels, the size YOLO11 was pretrained at

if BATCH_SIZE not in (8, 16, 32) or not 1 <= EPOCHS <= 100:
    raise ValueError("Choose BATCH_SIZE from 8, 16, 32 and EPOCHS between 1 and 100.")
print(f"Training for {EPOCHS} epochs with batches of {BATCH_SIZE} images at {IMG_SIZE} px.")
```

```
Training for 20 epochs with batches of 16 images at 640 px.
```

New

```
# ▸ 🔎 Plot the Training Curves
```



5 · TP, FP, FN kept; the IoU implementation removed

Your TP/FP/FN task stays, now with every box colored by the matching rule (green found, orange invented, red missed) and Previous / Next through the validation images. The IoU coding task is gone, as you suggested; Know Your Images has the optional IoU task.

Your YOLOv7 workbook

Check the labels and predictions of the sample image.

- What are true positives, false positives, and false negatives?
- Calculate the IOU and print it for the boxes that match with an IOU of at least 0.5

```
def calculate_iou(box1, box2):

    x_min1, y_min1 = box1[0]
    x_max1, y_max1 = box1[1]
    x_min2, y_min2 = box2[0]
    x_max2, y_max2 = box2[1]

    #TODO: continue here
    iou = 0

    return iou
```

New

🟢 True Positives, False Positives, False Negatives: What Went Wrong?

Each image shows the ground truth as dashed outlines and the predictions as solid boxes with class and confidence, colored by the rule above.

Before running the code:

- Which mistake do you expect more often: invented bunches (FP) or missed bunches (FN)?

Explore:

- Browse at least ten validation images (Previous / Next).

Observe and interpret:

- Find one example of each: a true positive, a false positive, a false negative.
- Why might the model have made each FP and FN? Consider object size, occlusion, the image edge, and whether the ground truth is right.

```
# ▸ 🔎 Browse Predictions vs. Ground Truth
```

CDY_2015.jpg · TP 19 · FP 4 · FN 5

true positive (found) · false positive (invented) · false negative (missed) · ground truth, matched (white)

6 · New: confidence threshold and precision–recall

A slider moves the confidence threshold; one image shows the TP/FP/FN at that threshold and the precision–recall curve shows where the threshold puts you. Shown here at 0.70.

New in this version

The Confidence Threshold: Precision or Recall?

The vineyard counts bunches to estimate the harvest. An invented box inflates the estimate; a missed bunch deflates it.

Before running the code:

- Raising the confidence threshold from 0.1 to 0.8: what happens to precision and recall? Write down your guess.

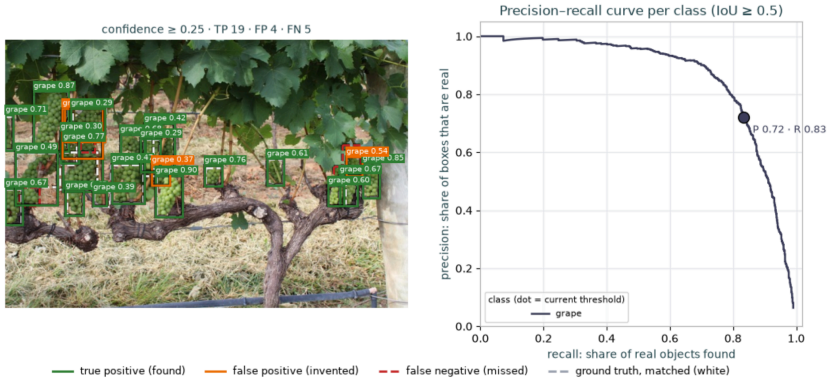
Explore:

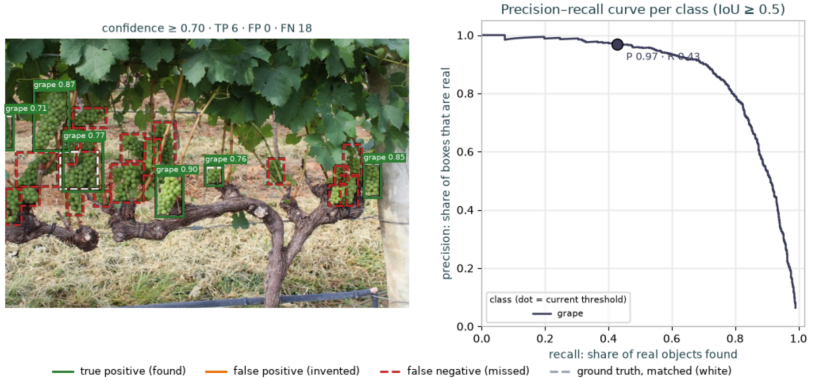
1. Move the `confidence` slider from low to high. The left panel shows one image at that threshold; the right shows each class's precision–recall curve, with a dot at the threshold.
2. Repeat on two or three other images with the `image` slider.

Observe and interpret:

- What happens to the orange (invented) and the red (missed) boxes as the threshold rises?
- Which threshold would you choose for the harvest estimate, and why is none best for both precision and recall?

```
# ▸ 🔎 Explore the Confidence Threshold
```





7 · New: AP, mAP@50 and mAP@50–95

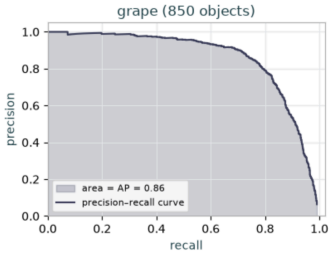
AP drawn as the shaded area under each class's curve, checked against Ultralytics' own value; then AP at the ten IoU thresholds whose mean is mAP@50–95, with a task on what each metric rewards.

New in this version

```
# ▸ 🔎 Compute AP per Class and Compare with Ultralytics
```

```
Ultralytics 8.4.208 🚀 Python-3.12.12 torch-2.15.0 MPS (Apple M2 Pro)
YOLO11n summary (fused): 101 layers, 2,597,051 parameters, 0 gradients, 6.4 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2857.0±664.0 MB/s, size: 126.0 KB)
val: Scanning /Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/data/datasets/wgisd-grapes-242-58/valid/labels.cach
e... 58 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 58/58 60.8Mit/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 4/4 5.3it/s 0.7s
                   all         58        850      0.837      0.762       0.86      0.537
Speed: 0.1ms preprocess, 6.0ms inference, 0.0ms loss, 2.8ms postprocess per image
```

AP is the shaded area under each class's curve (IoU ≥ 0.5)




```
mAP@50, our computation: 0.863 (mean of the class APs above)
mAP@50, Ultralytics val:  0.860 (small differences come from how boxes are matched)
```

🟡 mAP@50 vs. mAP@50–95: What Does Each Reward?

So far a prediction counted as correct at IoU ≥ 0.5, which a box covering two-thirds of an object can pass. **mAP@50–95** averages the AP at ten IoU thresholds, 0.50, 0.55, ..., 0.95. It is the main COCO metric, used in the model table at the top.

Before running the code:

- Will AP go up or down as the required IoU rises from 0.50 to 0.95?

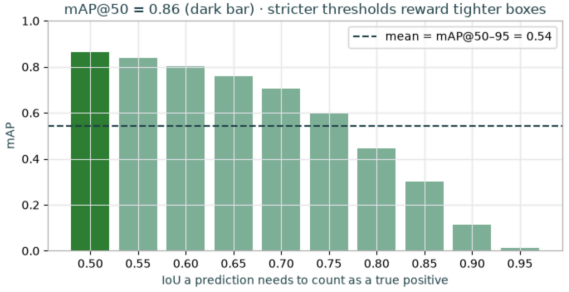
Explore:

- Run the cell. Each bar is the mAP at one IoU threshold; the dashed line is their mean, mAP@50–95.

Observe and interpret:

- How large is the gap between mAP@50 and mAP@50–95? What does it tell you about the boxes?
- Name one application where mAP@50 is the number to watch, and one where mAP@50–95 matters more.

```
# ▸ 🔎 AP at Ten IoU Thresholds
```




```
mAP@50-95, our computation: 0.544 · Ultralytics val: 0.537
```

8 · New: the NMS slider

The validation image with the most objects (27 grape bunches), at the default NMS IoU of 0.7 next to the student's setting. Shown at 0.95: duplicates appear (false positives 2 → 58). At 0.3 the two false positives go and no real bunch is lost, so the 'lost neighbors' side of NMS is explained in the Sample Answer rather than seen.

New in this version

```
# ▸ 🔎 Explore the NMS Threshold
```

9 · YOLOv7-tiny became YOLO11n vs YOLO11s (optional)

Both optional experiments stay behind one checkbox, so Run all stays fast. In our run YOLO11s was clearly better (mAP@50 0.90 vs 0.86, mAP@50–95 0.59 vs 0.54) and took about 1.5 times as long; freezing the backbone lowered mAP@50 to 0.81 and trained fastest.

Your YOLOv7 workbook

Optional Exercise: Download and Train Tiny YOLO Model

```
# download trained weights, only needed once or upon updates
!wget https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-tiny.pt
```

Exercise

Check individual predictions. The results can be found in the `runs/detect/...` folder of `YOLO`.

```
!python train.py --device 0 --workers 8 --batch-size 16 --epochs 200 \
 --img 640 640 --data data/cattle/cattle.yaml \
 --hyp data/cattle/hyp.scratch.cattle.yaml \
 --cfg cfg/training/yolov7-tiny.yaml \
 --weights yolov7-tiny.pt --name yolov7-tiny-cattle
```

New

YOLO11n vs. YOLO11s: Is the Larger Model Worth It? (Optional)

YOLO11s has about 3.6 times as many parameters as YOLO11n.

Before running the code:

- How much better do you expect YOLO11s to be on our small dataset, and how much slower?

Explore:

- Run the cell. It trains YOLO11s with your settings and compares both models on the validation images.

Observe and interpret:

- Compare mAP@50, mAP@50–95, training time and prediction time per image.
- For which use would you choose each model?

```
# ▸ 🔎 Train YOLO11s and Compare
```

```
Ultralytics 8.4.208 🚀 Python-3.12.12 torch-2.15.0 MPS (Apple M2 Pro)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7.5, cache=F
alse, cfg=None, channels_last=None, classes=None, close_mosaic=10, cls=0.5, cls_pw=0.0, cls_remap=True, compile=False, conf=None, cop
y_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=/Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/
data/datasets/wgisd-grapes-242-58/data.yaml, degrees=0.0, deterministic=True, device=mps, dfl=1.5, dgrad=0.5, dis=6.0, distill_model=
None, dlam=1.0, dlog=1.0, dnn=False, dropout=0.0, dynamic=False, embed=None, epochs=20, erasing=0.4, exist_ok=True, fliplr=0.5, flipu
d=0.0, format=torchscript, fraction=1.0, freeze=None, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, iou=None, kobj=1.0, line_width=No
ne, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolo11s.pt, momentum=0.937, mosaic=1.0, multi_scale=
0.0, name=yolo11s, nbs=64, nms=None, opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, pl
ots=True, pose=12.0, pretrained=True, profile=False, project=/Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/runs,
quantize=None, rect=False, resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False, save_crop=False, save_dir=/Users/si
mone.bonato/Work/notebooks/yolo-object-detection-notebook/runs/yolo11s, save_frames=False, save_json=False, save_period=-1, save_txt=
False, scale=0.5, seed=42, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False,
source=None, split=val, stream_buffer=False, task=detect, tile=False, time=None, tracker=tracktrack.yaml, translate=0.1, val=True, ve
rbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=
2, workspace=None
Overriding model.yaml nc=80 with nc=1

                   from  n    params  module                                       arguments
  0                  -1  1       928  ultralytics.nn.modules.conv.Conv             [3, 32, 3, 2]
  1                  -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 2]
  2                  -1  1     26080  ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
  3                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
  4                  -1  1    103360  ultralytics.nn.modules.block.C3k2            [128, 256, 1, False, 0.25]
  5                  -1  1    590336  ultralytics.nn.modules.conv.Conv             [256, 256, 3, 2]
  6                  -1  1    346112  ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
  7                  -1  1   1180672  ultralytics.nn.modules.conv.Conv             [256, 512, 3, 2]
  8                  -1  1   1380352  ultralytics.nn.modules.block.C3k2            [512, 512, 1, True]
  9                  -1  1    656896  ultralytics.nn.modules.block.SPPF            [512, 512, 5]
 10                  -1  1    990976  ultralytics.nn.modules.block.C2PSA           [512, 512, 1]
 11                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 12             [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 13                  -1  1    443776  ultralytics.nn.modules.block.C3k2            [768, 256, 1, False]
 14                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 15             [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 16                  -1  1    127680  ultralytics.nn.modules.block.C3k2            [512, 128, 1, False]
 17                  -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
 18            [-1, 13]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 19                  -1  1    345472  ultralytics.nn.modules.block.C3k2            [384, 256, 1, False]
 20                  -1  1    590336  ultralytics.nn.modules.conv.Conv             [256, 256, 3, 2]
 21            [-1, 10]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 22                  -1  1   1511424  ultralytics.nn.modules.block.C3k2            [768, 512, 1, True]
 23        [16, 19, 22]  1    819795  ultralytics.nn.modules.head.Detect           [1, 16, None, [128, 256, 512]]
YOLO11s summary: 181 layers, 9,428,179 parameters, 9,428,163 gradients, 21.7 GFLOPs

Transferred 493/499 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
train: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2157.9±422.7 MB/s, size: 132.8 KB)
train: Scanning /Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/data/datasets/wgisd-grapes-242-58/train/labels.cac
he... 242 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 242/242 253.8Mit/s 0.0s
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 3233.1±607.8 MB/s, size: 123.7 KB)
val: Scanning /Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/data/datasets/wgisd-grapes-242-58/valid/labels.cach
e... 58 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 58/58 81.1Mit/s 0.0s
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and determining best 'optimizer' and 'lr0' automatically...
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Plotting labels to /Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/runs/yolo11s/labels.jpg...
Using 242 train, 58 val images for fraction=1.0 at imgsz=640
Using 0 dataloader workers
Logging results to /Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/runs/yolo11s
Starting training for 20 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
/Users/simone.bonato/Work/notebooks/yolo-object-detection-notebook/.venv/lib/python3.12/site-packages/torch/autograd/graph.py:1113: U
serWarning: scatter_reduce_mps does not have a deterministic implementation, but you set 'torch.use_deterministic_algorithms(True, wa
rn_only=True)'. You can file an issue at https://github.com/pytorch/pytorch/issues to help us prioritize adding deterministic support
for this operation. (Triggered internally at /Users/runner/work/pytorch/pytorch/aten/src/ATen/Context.cpp:224.)
```